Property C: The LLM Does Not Hold Substrate-Relevant State Outside Substrate — Standalone Architectural Commitment in the CKS Pattern

Author: Wenxin Li (Independent Researcher) **ORCID:** 0009-0004-8065-3235 **Date:** 2 May 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the five mediator properties named in the source paper's "AI-as-substrate-mediator" commitment — **Property C**, that the LLM does not hold substrate-relevant state outside the substrate — as a standalone architectural commitment with independent operational content, separable from the other four properties with which it composes into the integrated mediator role.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's "AI-as-substrate-mediator" commitment names five properties — substrate-content reads as primary state (A), substrate-content writes under orchestration rules (B), no substrate-relevant state outside substrate (C), no authority over substrate content (D), and LLM writes recorded with attribution (E) — as jointly necessary for the architectural role the term carries. A separate parent-frame note formalizes the joint commitment as a severable set of five. This note formalizes Property C as having independent operational content, separately defensible, implementable, and testable from the other four. The motivation is concrete: the dominant LLM deployment patterns in 2024–2026 — agent frameworks with persistent memory, conversational interfaces with thread-bound history, fine-tuning of models on substrate-derived content, retrieval pipelines with persistent caches — all drift toward holding substrate-relevant state in LLM-internal locations, producing the failure mode the source paper at §6.2 names *context rot*. Property C is the direct architectural prevention of that failure. The note states the five operational components of the commitment, distinguishes it from four adjacent patterns commonly conflated with it, traces five load-bearing connections to other CKS commitments, enumerates eight failure modes that violate it specifically, and provides an operational test for whether a system's LLM state behavior is CKS-coherent on this axis.

1. Why Property C needs to be formalized as standalone

The parent foundational treatment commits to AI-as-substrate-mediator as a five-property architectural role. The parent-frame decomposition note formalizes the five as a severable set — each with independent content, jointly composing into the integrated role but separately defensible. This note formalizes the third, Property C: the LLM does not hold substrate-relevant state in any location external to the substrate.

The motivating cases are implementations where the LLM accumulates substrate-relevant state in LLM-internal locations. Agent-framework deployments maintain memory layers persisting across tasks, holding decision history, learned preferences, and accumulated context the agent consults as authoritative. Chat-application deployments treat prior conversation turns as the state-of-record, with substrate writes (if any) lagging behind. Fine-tuning deployments train models on substrate-derived content periodically, after which the model's parametric memory is consulted as authoritative for that content. Persistent context-window deployments accumulate substrate-relevant content across sessions. Each pattern produces the failure mode the source paper at §6.2 names *context rot*: state migrates externally, degrades or fragments over time, and continues to be consulted as authoritative until divergence between it and the substrate becomes operationally unmanageable.

Two further motivations matter. The first is strategic prior-art posture: any LLM state-management architecture — agent-memory design, context-window strategy, fine-tuning approach, persistent-cache scheme — is more defensibly contested when Property C is publicly formalized as standalone, because each can be evaluated against the specific commitment to no-substrate-relevant-state-outside-substrate. The second is cost-model dependency: the linear-cost commitment depends partially on Property C, because per-execution LLM cost stays bounded only if the LLM does not accumulate substrate-relevant state externally. External accumulation would scale cost with accumulated size, breaking the cost contract §6 of the source paper commits to.

2. The Property C commitment, defined precisely

In the CKS pattern, an LLM operating within a cell satisfies **Property C** if and only if all of the following five conditions hold during cell execution.

(a) No agent-framework memory holding substrate-relevant state across cell executions. State that affects coordination questions, decisions about substrate writes, or interpretations of substrate content lives in the substrate, not in agent-framework memory layers, per-agent storage, or any cross-execution location the framework provides. The framework may be a useful adjacent technology; what cannot happen is its memory becoming a substrate-relevant state location outside substrate.

(b) No persistent context window holding substrate-relevant state across cell executions. Each execution begins with fresh context populated by what the orchestration rule specifies and what the cell reads from substrate at execution time. Context persisting from prior executions and containing substrate-relevant content violates the commitment, regardless of implementation — literal context concatenation, retrieval-based reconstruction with persistent indices treated as authoritative, conversation-thread continuation.

(c) No session-bound storage holding substrate-relevant state across executions. Browser sessions, application sessions, conversation threads, and user-specific stores that hold substrate-relevant content across interactions violate the commitment, even when the storage is associated with a "session" rather than the LLM directly. The architectural property is *where the state lives*, not what the storage is named.

(d) No fine-tuned weights treated as authoritative for substrate content. Fine-tuning may be used as deployment technology — for style, tool-use protocols, domain vocabulary. What cannot happen is

the fine-tuned model treating substrate-derived training content as a substitute for substrate. Substrate content the LLM consults must come from substrate at execution time, not from parametric memory of past substrate content.

(e) No persistent cache of substrate content consulted as authoritative across executions. An execution-scoped cache existing only during a single invocation is admissible (cell-internal state). A persistent cache accumulating substrate content across executions and consulted as authoritative violates the commitment. A cache derived from substrate, refreshed under known invalidation conditions, and treated as non-authoritative may coexist with substrate; a cache treated as the answer to "what is the case" does not.

A system that satisfies fewer than five holds substrate-relevant state in LLM-internal locations in some way, breaking the architectural commitment.

3. What Property C does NOT require

Stating precisely what Property C does not require keeps the standalone treatment from drifting into something stronger than the source paper supports.

It does not require the LLM to be technically stateless during a single cell execution. LLMs maintain context, intermediate reasoning, and execution state during inference; the commitment is about state that persists across executions, not state that exists during one. A cell whose execution begins with fresh context, processes through the orchestration rule's logic, produces outputs, and completes without holding substrate-relevant state into the next invocation satisfies Property C — even with substantial internal state during the invocation itself.

It does not require the LLM to lack parametric world knowledge. Training-derived knowledge — language, common-sense reasoning, domain vocabulary — is part of what makes the LLM useful as a mediator. This parametric knowledge is not substrate-relevant; it is general knowledge the LLM uses to reason over substrate content. The commitment forbids substrate-relevant state externally, not all external state.

It does not forbid execution-scoped caches. A cell may cache intermediate substrate reads, derived computations, or working content during its execution. These are cell-internal state and are released when the execution completes.

It does not forbid orchestration-rule content from appearing in LLM context. Orchestration rules are substrate content; they are read by cells and may appear in the LLM's prompt or context for the specific execution. The rule content in context is not a violation; what would be a violation is rule content persisting in LLM context across executions independently of substrate reads.

It does not forbid deployment configurations. Model selection, runtime parameters, cell orchestration settings — these are deployment decisions that affect how cells execute, not coordination state that affects what cells decide.

4. What Property C is NOT

Four adjacent patterns are commonly conflated with Property C. Each is a real and reasonable commitment in some other architecture; naming what Property C is not is what prevents the misreading.

Not a prohibition on agent memory in general. This is the most consequential conflation in 2024–2026 LLM infrastructure, because agent frameworks are the dominant deployment pattern and their memory layers serve heterogeneous purposes that vary in their substrate-relevance. Frameworks use memory layers for tracking conversation state across multi-turn interactions, holding task-decomposition plans, recording tool-call histories, supporting reasoning scratchpads within a single complex task, and maintaining learned preferences or user models across sessions. Some of these are not substrate-relevant in the architectural sense — a reasoning scratchpad holding intermediate inferences during a single cell invocation is cell-internal state, not external substrate-relevant state. Others are substrate-relevant — a learned-preferences store influencing coordination decisions across executions is substrate-relevant state held outside substrate, regardless of whether the framework calls it "memory" or "configuration." Property C specifically prohibits the substrate-relevant uses; it does not prohibit agent frameworks as deployment technology, nor memory layers used for non-substrate-relevant purposes within a single execution. A CKS deployment using an agent framework can use the framework's memory for non-substrate-relevant purposes while satisfying Property C, provided the substrate-relevant state lives in substrate.

Not a prohibition on context windows. LLMs have context windows; the commitment is that context content persisting across executions and containing substrate-relevant state violates Property C. A cell whose execution begins with fresh context populated by substrate reads, processes through, and completes — even with a large amount of substrate content during the execution — is fully Property C-coherent. Long context windows are not violations; *persistent* context windows accumulating substrate-relevant content across executions are.

Not a prohibition on fine-tuning. LLMs may be fine-tuned for domain content, coordination patterns, organizational style, or tool-use protocols. The fine-tuning is not a violation; what would violate Property C is treating fine-tuned weights as authoritative for substrate content. A fine-tuned LLM that operates over substrate as primary source (Property A), writes under orchestration rules (Property B), and does not treat its fine-tuned knowledge as a substitute for substrate satisfies Property C.

Not a prohibition on retrieval. RAG, vector search, knowledge graphs — the architecture supports these. What Property C forbids is retrieval results held as substrate-relevant state across executions. Single-execution retrieval is admissible; retrieval results accumulated in LLM-side memory across executions and treated as authoritative violate the commitment. A retrieval index is permissible as adjacent representation when derived from and reconcilable to substrate; it is a Property-C violation when it has become a substitute source-of-truth.

5. Why Property C is load-bearing for downstream commitments

Property C has direct connections to five other CKS commitments.

Source-of-truth. Property C jointly with Property A is what makes substrate's authoritative status architecturally specific: the LLM reads from substrate (A) and does not hold substrate-relevant state externally (C). Without C, A is satisfiable in form while substrate authority is undermined in practice — the LLM "reads from substrate" but consults its agent memory, persistent context, or fine-tuned weights as the actual answer.

Linear-cost scaling. Per-execution cost stays bounded because the LLM does not maintain substrate-relevant state across executions. If state accumulated externally, per-execution cost would scale with accumulated size — context windows growing, retrieval indices growing, agent-memory queries growing — and the linear-cost commitment would break.

Substrate-cell boundary. Cells are stateless across executions. The LLM operating within a cell inherits this statelessness on the substrate-relevant axis through Property C: the cell cannot hold substrate-relevant state externally because the LLM operating within it cannot.

Cell-internal vs. substrate state distinction. The broader distinction places coordination state in substrate and execution state in cells. Property C is the LLM-specific specialization: the LLM's execution state during a single invocation is cell-internal (released after execution); the LLM's substrate-relevant state across executions is in substrate.

Context-rot prevention. §6.2 of the source paper names the failure mode that arises when state migrates into LLM-internal storage, accumulates, degrades, and continues to be consulted as authoritative. Property C is the direct architectural prevention of this failure mode. The other four mediator properties prevent different failure modes; Property C is specifically the no-context-rot commitment.

6. Failure modes that violate Property C

A system can fail Property C specifically, even when it satisfies the other four mediator properties. Eight failure modes name the most common patterns.

(a) Agent-framework memory holding substrate-relevant state. The LLM operates within a framework whose memory persists across executions, holding coordination state, decision history, or interpretations of substrate content treated as authoritative; substrate is consulted as backup. This is the canonical failure mode in 2024–2026 deployments and the most consequential pattern Property C rules out.

(b) Persistent context windows. The context window persists across executions, accumulating substrate-relevant content. Subsequent executions read from the persistent context as authoritative; substrate is consulted only when the context lacks what is needed. The persistent context becomes a competing source of truth.

(c) Session-bound substrate-relevant state. The application's session layer (browser session, conversation thread) holds substrate-relevant state across user interactions and is treated as authoritative for the session's coordination work. Although session-bound rather than LLM-bound, it functions as substrate-relevant state outside substrate.

(d) Fine-tuned weights treated as authoritative. Models are fine-tuned on substrate content periodically; subsequent inference treats the fine-tuned knowledge as authoritative for that content. The failure is in treating fine-tuned knowledge as a substitute for substrate rather than as a deployment optimization.

(e) Persistent caches of substrate content treated as authoritative. A cache survives across executions and is consulted as authoritative, with substrate read only on cache miss. What started as a performance optimization has become a substrate-relevant state location outside substrate.

(f) Learned preferences or user models outside substrate. Models of preferences, organizational style, or coordination patterns influence LLM behavior across executions; they hold substrate-relevant state and are external to substrate. The commitment is violated even when human-supervised; the architectural property is *where the state lives*.

(g) Conversation history as state-of-record. In chat-style deployments, conversation history holds the coordination state; new turns reference history as authoritative. Substrate writes happen, but the conversation, not substrate, is the authoritative source for what was discussed and decided.

(h) Implicit state through prompt engineering. Prompt patterns implicitly carry substrate-relevant state — system prompts containing organizational context, user roles, coordination policies. This implicit state lives in the prompt design (outside substrate as code or configuration); the LLM operates over it as if authoritative.

A system exhibiting any of (a)–(h) does not satisfy Property C, even if its other mediator properties are preserved.

7. Operational test

A system satisfies Property C if and only if all of the following are true at all times during LLM execution within cells.

1. No agent-framework memory layer holds substrate-relevant state across cell executions. 2. No persistent context window holds substrate-relevant state across cell executions; each execution begins with fresh context populated from substrate and orchestration rules at execution time. 3. No session-bound or application-bound storage layer holds substrate-relevant state across executions. 4. Fine-tuned model weights, if used, are not treated as authoritative for substrate content; substrate content consulted by the LLM comes from substrate at execution time, not from parametric memory of past substrate content. 5. Caches of substrate content, if used, are scoped to single cell executions; persistent caches consulted as authoritative across executions are not present. 6. No external models, learned preferences, or implicit state mechanisms (including system-prompt-embedded coordination policies) hold substrate-relevant state across executions; coordination state lives in substrate.

A system that fails any of (1)–(6) does not satisfy Property C in the architectural sense, even if its other mediator properties are preserved. Such a system may instantiate some other LLM-state architecture, but is not CKS-coherent on the storage-direction axis.

8. Why naming Property C as standalone matters

Implementations under pressure to support agentic LLM behavior, conversational interfaces, or personalized AI consistently drift toward holding substrate-relevant state externally. The drift is steady because external state is architecturally familiar from agent frameworks and chat applications, because it offers performance and personalization benefits substrate-only state does not, and because the difference between "substrate-relevant" and "non-substrate-relevant" external state requires explicit architectural attention to maintain.

Implementations drifting away from Property C produce systems where substrate exists but coordination state has migrated externally — agent memory, conversation history, cached views, fine-tuned models. The downstream consequences manifest as the context-rot failure mode §6.2 of the source paper names: external state degrades, fragments, or loses synchronization with substrate while continuing to be consulted as authoritative. The system appears to function until it doesn't, and recovery then requires reconstructing substrate state from the degraded external copies — a salvage operation rather than a routine read.

Naming Property C as standalone — with the five components in §2, the limitations in §3, the four adjacent-pattern distinctions in §4, the load-bearing connections in §5, and the eight failure modes in §6 — gives downstream implementers a precise specification of what Property C requires, separable from the other four mediator properties. Subsequent decomposition notes specialize the remaining two (Property D on no authority over substrate content; Property E on attribution of LLM-authored writes), each receiving similar standalone prior-art treatment. Subsequent work that adopts, extends, composes, or argues against the CKS AI-as-substrate-mediator commitment should use Property C in the sense formalized here. Subsequent work that holds substrate-relevant state in LLM-internal locations is using a different architectural pattern, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Property C: The LLM Does Not Hold Substrate-Relevant State Outside Substrate — Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern.* 2 May 2026. ORCID: 0009-0004-8065-3235.